

TP 1 : Introduction à la programmation orientée objet en Python

Corrigé détaillé et commenté

Ce document présente une correction complète du TP 1. Pour chaque question, on donne le code Python demandé puis on explique les choix d'implémentation. Les trois exercices sont traités dans l'ordre de l'énoncé : la classe `CompteBancaire`, la classe `Date` avec validation, puis la classe `Etudiant` et le calcul de moyenne pondérée.

Exercice 1 — Compte bancaire

Question 1 — Attributs de classe ou attributs d'instance ?

Les trois attributs (nom de la banque, nom du titulaire, solde) doivent être déclarés comme des **attributs d'instance**.

Justification. Un **attribut de classe** n'existe qu'en un seul exemplaire : il est partagé par toutes les instances et possède la même valeur pour tous les objets. Un **attribut d'instance**, au contraire, appartient à un objet précis : chaque objet en possède sa propre copie, créée dans le constructeur via `self`. Or chaque compte bancaire a sa propre banque, son propre titulaire et son propre solde : deux comptes différents ont en général des valeurs différentes pour ces trois informations. Elles doivent donc être stockées séparément dans chaque objet, ce qui correspond exactement à la définition d'un attribut d'instance.

Remarque. Si l'énoncé imposait que tous les comptes appartiennent à une même banque, `nomBanque` aurait pu être un attribut de classe (valeur commune). Ce n'est pas le cas ici.

Question 2 — Constructeur avec validation des données

En Python, la manière propre d'empêcher la création d'un objet lorsque les données sont invalides consiste à **lever une exception** (`raise`) dans le constructeur `__init__`. Dès qu'une exception est levée, l'exécution du constructeur s'arrête : l'affectation `c = CompteBancaire(...)` échoue et aucun objet n'est renvoyé au programme.

```
class CompteBancaire:

    def __init__(self, nomBanque, nom, solde):
        # 1) Les noms ne doivent pas être vides
        if not isinstance(nomBanque, str) or nomBanque.strip() == "":
            raise ValueError("Le nom de la banque ne doit pas être vide.")
        if not isinstance(nom, str) or nom.strip() == "":
            raise ValueError("Le nom du titulaire ne doit pas être vide.")
        # 2) Le solde initial doit être un nombre positif ou nul
        if not isinstance(solde, (int, float)) or solde < 0:
            raise ValueError("Le solde initial ne peut pas être négatif.")
        # 3) Données valides : on crée les attributs d'instance
        self.nomBanque = nomBanque
        self.nom = nom
        self.solde = solde
```

Explications. `nomBanque.strip() == ""` détecte aussi bien la chaîne vide qu'une chaîne composée uniquement d'espaces ; `isinstance(...)` vérifie le type avant toute comparaison, ce qui évite un plantage si l'on passe par exemple `None` ; enfin `ValueError` est l'exception standard pour signaler une valeur incorrecte. À l'utilisation, on entoure la création d'un bloc `try / except` :

```
try:
    c1 = CompteBancaire("Banque Populaire", "Med", 1000)    # acceptée
    c2 = CompteBancaire("", "Ali", -50)                      # refusée
except ValueError as e:
    print("Création refusée :", e)
```

Question 3 — Méthode déposer

```
def depoter(self, montant):
    """Ajoute une somme strictement positive au solde."""
    if montant <= 0:
        print("Dépôt refusé : le montant doit être strictement positif.")
        return False
    self.solde += montant
    return True
```

La condition `montant <= 0` rejette à la fois les montants nuls et négatifs, comme l'exige l'énoncé. `self.solde += montant` est un raccourci de `self.solde = self.solde + montant`. La méthode renvoie un booléen (`True` si l'opération a réussi, `False` sinon) : le programme appelant peut ainsi savoir si le dépôt a été accepté.

Question 4 — Méthode retirer

```
def retirer(self, montant):
    """Retire une somme positive si le solde est suffisant."""
    if montant <= 0:
        print("Retrait refusé : le montant doit être strictement positif.")
        return False
    if montant > self.solde:
        print("Retrait refusé : solde insuffisant.")
        return False
    self.solde -= montant
    return True
```

Deux vérifications sont nécessaires : le montant doit être strictement positif, et il ne doit pas dépasser le solde disponible. Comme le retrait n'est effectué que lorsque `montant <= self.solde`, le solde reste toujours positif ou nul : la règle « aucun découvert » est garantie par construction.

Question 5 — Méthode afficher

```
def afficher(self):
    """Affiche l'état du compte."""
    print("Banque :", self.nomBanque)
    print("Titulaire :", self.nom)
    print("Solde :", self.solde)
```

On peut aussi utiliser les f-strings, plus lisibles lorsqu'on mélange texte et variables : `print(f"Banque : {self.nomBanque}")`. Petit programme de test regroupant toutes les méthodes :

```
c = CompteBancaire("Banque Populaire", "Med", 1000)
c.deposer(500)      # solde : 1500
c.retirer(200)      # solde : 1300
c.retirer(5000)     # refusé : solde insuffisant
c.deposer(-10)      # refusé : montant non positif
c.afficher()
```

Affichage obtenu :

```
Retrait refusé : solde insuffisant.
Dépôt refusé : le montant doit être strictement positif.
Banque : Banque Populaire
Titulaire : Med
Solde : 1300
```

Question 6 — Trier listComptes par nom de titulaire

La méthode `sort()` de la classe `list` accepte un paramètre optionnel `key` : une fonction qui, pour chaque élément de la liste, renvoie la valeur à utiliser comme critère de tri. Il suffit donc de lui fournir une fonction qui, à partir d'un compte, renvoie le nom de son titulaire :

```
listComptes.sort(key=lambda compte: compte.nom)
```

`lambda compte: compte.nom` est une fonction anonyme : elle reçoit un objet `CompteBancaire` et renvoie son attribut `nom`. `sort()` trie alors les comptes selon l'ordre alphabétique de ces noms, directement « en place » (la liste elle-même est modifiée). Pour conserver la liste d'origine intacte, on utilise plutôt la fonction `sorted()`, qui renvoie une nouvelle liste triée :

```
listTrie = sorted(listComptes, key=lambda compte: compte.nom)
```

Remarque. Pour un tri insensible à la casse (majuscules/minuscules), on écrit `key=lambda compte: compte.nom.lower()`.

Exercice 2 — Une classe Date avec validation

Question 1 — Méthode de classe estBissextile

Le caractère bissextile ne dépend d'aucune date particulière : c'est une propriété de l'année elle-même. Il est donc naturel d'en faire une **méthode de classe**, déclarée avec le décorateur `@classmethod`. Son premier paramètre, nommé `cls` par convention, désigne la classe (et non une instance) ; on peut ainsi l'appeler sans créer d'objet, par exemple `Date.estBissextile(2024)`.

```
class Date:

    @classmethod
    def estBissextile(cls, y):
        """Renvoie True si l'année y est bissextile."""
        return (y % 4 == 0 and y % 100 != 0) or (y % 400 == 0)
```

Le code traduit littéralement la règle de l'énoncé : divisible par 4 **et** non divisible par 100, **ou** divisible par 400. L'opérateur `%` (modulo) donne le reste de la division entière : `y % 4 == 0` signifie « y est divisible par 4 ». Exemples : 2024 est bissextile (divisible par 4, pas par 100) ; 1900 ne l'est pas (divisible par 100 mais pas par 400) ; 2000 l'est (divisible par 400).

On ajoute dès maintenant une seconde méthode de classe, `nbJours(m, y)`, qui renvoie le nombre de jours du mois `m` de l'année `y` : elle servira au constructeur (question 2) et au calcul d'écart entre dates (question 4).

```
@classmethod
def nbJours(cls, m, y):
    """Nombre de jours du mois m de l'année y."""
    if m in (1, 3, 5, 7, 8, 10, 12):
        return 31
    elif m in (4, 6, 9, 11):
        return 30
    else:
        # février
        return 29 if cls.estBissextile(y) else 28
```

Question 2 — Constructeur de la classe Date

Comme dans l'exercice 1, une valeur invalide déclenche une exception et l'objet n'est pas créé :

```
def __init__(self, y, m, d):
    if y <= 0:
        raise ValueError("L'année doit être strictement positive.")
    if not (1 <= m <= 12):
        raise ValueError("Le mois doit être compris entre 1 et 12.")
    if not (1 <= d <= Date.nbJours(m, y)):
        raise ValueError("Jour invalide pour ce mois et cette année.")
    self.y = y
    self.m = m
    self.d = d
```

L'ordre des vérifications est important : la validité du jour dépend du mois et de l'année (à cause de février et des années bissextiles). On contrôle donc d'abord `y` puis `m` avant d'appeler `Date.nbJours(m, y)`. Ainsi `Date(2023, 2, 29)` est refusée (2023 n'est pas bissextile, février n'a que 28 jours) alors que `Date(2024, 2, 29)` est acceptée.

Question 3 — Comparer la date locale avec une date d

On compare deux dates comme des mots dans un dictionnaire : d'abord les années ; en cas d'égalité, les mois ; puis les jours. La méthode renvoie un code conventionnel : `-1` si la date locale (`self`) est **antérieure** à `d`, `1` si elle est **postérieure**, `0` si les deux dates sont identiques.

```
def comparer(self, d):
    """-1 : self antérieure à d ; 1 : postérieure ; 0 : égales."""
    if self.y != d.y:
        return -1 if self.y < d.y else 1
    if self.m != d.m:
        return -1 if self.m < d.m else 1
    if self.d != d.d:
        return -1 if self.d < d.d else 1
    return 0
```

Remarque. En Python, les tuples se comparent exactement de cette manière (élément par élément, de gauche à droite). On peut donc écrire la même méthode de façon plus compacte en comparant `(self.y, self.m, self.d)` avec `(d.y, d.m, d.d)`.

Question 4 — Nombre de jours entre deux dates

L'idée la plus simple consiste à convertir chaque date en un nombre unique : son **rang**, c'est-à-dire le nombre de jours écoulés depuis une origine fixe (ici le 01/01/0001, qui porte le rang 1). L'écart entre deux dates est alors la différence de leurs rangs, prise en valeur absolue.

```
def versJours(self):
    """Rang de la date : nombre de jours depuis le 01/01/0001."""
    total = self.d
    # + les jours des mois déjà écoulés dans l'année courante
    for m in range(1, self.m):
        total += Date.nbJours(m, self.y)
    # + les jours des années complètes précédentes
    for y in range(1, self.y):
        total += 366 if Date.estBissextile(y) else 365
    return total

def difference(self, d):
    """Nombre de jours séparant la date locale de la date d."""
    return abs(self.versJours() - d.versJours())
```

versJours additionne trois contributions : le jour dans le mois courant, les jours des mois précédents de la même année, et les jours de toutes les années antérieures (365 ou 366 selon qu'elles sont bissextiles ou non). **abs()** rend le résultat positif quel que soit l'ordre des deux dates. Exemple :

```
d1 = Date(2026, 7, 18)
d2 = Date(2026, 1, 1)
print(d1.comparer(d2))      # 1   (d1 est postérieure à d2)
print(d1.difference(d2))    # 198 (jours entre les deux dates)
```

Remarque. La boucle sur les années peut être remplacée par la formule directe $365 * (y-1) + (y-1) // 4 - (y-1) // 100 + (y-1) // 400$, qui compte les années bissextiles antérieures sans boucle : le calcul devient immédiat même pour de grandes années.

Exercice 3 — Calcul de moyenne (classe Etudiant)

Le point 1 de l'énoncé décrit le contexte : chaque étudiant possède une note dans chacun des huit modules du tableau 1, ainsi qu'un numéro d'étudiant, un nom et un prénom. Les questions commencent au point 2.

Question 2 — Attributs de la classe Etudiant

Il faut distinguer ce qui est **commun à tous les étudiants** de ce qui est **propre à chacun** :

Attribut	Nature	Justification
coeff	attribut de classe	Dictionnaire {code du module : coefficient}. Les coefficients sont fixés par la formation : ils sont identiques pour tous les étudiants et doivent être partagés. Si un coefficient change, il change pour tout le monde.
numero	attribut d'instance	Numéro d'étudiant, propre à chaque étudiant.
nom, prenom	attributs d'instance	Identité propre à chaque étudiant.
notes	attribut d'instance	Dictionnaire {code du module : note} : chaque étudiant possède ses propres notes.

Le choix d'un **dictionnaire** pour les notes est naturel : il associe directement chaque module (la clé) à la note obtenue (la valeur), ce qui facilitera le calcul de la moyenne pondérée.

Question 3 — Constructeur de la classe Etudiant

```
class Etudiant:

    # Attribut de classe : coefficients communs à toute la formation
    coeff = {"Res": 3, "Admin": 1.5, "Bd": 1.5, "Web": 1.5,
            "tn": 3, "ang": 3, "com": 2, "prog": 1.5}

    def __init__(self, numero, nom, prenom):
        self.numero = numero          # attributs d'instance
        self.nom = nom
        self.prenom = prenom
        self.notes = {}               # {code du module : note}
```

Remarque. Piège classique : le dictionnaire `notes` doit impérativement être créé dans `__init__` (attribut d'instance). S'il était déclaré au niveau de la classe, tous les étudiants partageraient le même dictionnaire, et ajouter une note à un étudiant modifierait les notes de tous les autres.

Question 4 — Méthode addNote

```
def addNote(self, module, note):
    """Affecte la note d'un module à l'étudiant."""
    if module not in Etudiant.coeff:
        print("Module inconnu :", module)
        return False
    if not (0 <= note <= 20):
        print("Note invalide : elle doit être comprise entre 0 et 20.")
        return False
    self.notes[module] = note
    return True
```

On vérifie d'abord que le code du module figure bien parmi les modules de la formation (les clés de `Etudiant.coeff`), puis que la note est comprise entre 0 et 20. L'écriture `self.notes[module] = note` ajoute la note si le module n'était pas encore noté, ou remplace l'ancienne note sinon.

Question 5 — Méthode setCoeff

Les coefficients sont stockés dans un **attribut de classe** : les modifier doit affecter tous les étudiants à la fois. `setCoeff` est donc une **méthode de classe**, et la modification s'effectue sur `cls.coeff` :

```
@classmethod
def setCoeff(cls, module, valeur):
    """Change le coefficient d'un module (1.5, 2, 3 ou 4)."""
    if valeur not in (1.5, 2, 3, 4):
        print("Coefficient refusé : valeurs autorisées 1.5, 2, 3, 4.")
        return False
    if module not in cls.coeff:
        print("Module inconnu :", module)
        return False
    cls.coeff[module] = valeur
    return True
```

Le test `valeur not in (1.5, 2, 3, 4)` impose exactement l'ensemble de valeurs autorisé par l'énoncé. Comme `3 == 3.0` en Python, l'appel `Etudiant.setCoeff("Res", 3.0)` est lui aussi accepté.

Question 6 — Méthode moyenne

La moyenne demandée est une **moyenne pondérée** : la somme des produits (note × coefficient), divisée par la somme des coefficients.

```
def moyenne(self):
    """Moyenne pondérée des notes de l'étudiant."""
    somme_notes = 0
    somme_coeff = 0
    for module, note in self.notes.items():
        c = Etudiant.coeff[module]
        somme_notes += note * c
        somme_coeff += c
    if somme_coeff == 0:          # aucune note saisie
        return 0
    return somme_notes / somme_coeff
```

La boucle `for module, note in self.notes.items()` parcourt en même temps les clés (codes des modules) et les valeurs (notes) du dictionnaire. Pour chaque module, on récupère son coefficient dans l'attribut de classe. Le test `somme_coeff == 0` évite une division par zéro si l'étudiant n'a encore aucune note. Avec les huit modules du tableau 1, la somme des coefficients vaut $3 + 1.5 + 1.5 + 1.5 + 3 + 3 + 2 + 1.5 = 17$.

Question 7 — Trier la liste L (moyenne décroissante, puis nom)

Le tri porte sur **deux critères** : la moyenne, en ordre décroissant, puis le nom, en ordre alphabétique croissant en cas d'égalité des moyennes. Deux solutions complémentaires :

Solution 1 — une clé de tri composée. Comme à l'exercice 1, on utilise le paramètre `key` de `sort()`, mais en renvoyant cette fois un tuple :

```
L.sort(key=lambda e: (-e.moyenne(), e.nom))
for e in L:
    print(e.nom, "-", round(e.moyenne(), 2))
```

Python compare les tuples élément par élément : d'abord `-e.moyenne()`, puis `e.nom`. Le signe moins transforme le tri croissant (comportement par défaut de `sort`) en tri **décroissant** sur la moyenne ; à moyennes égales, les étudiants sont départagés par leur nom, trié alphabétiquement.

Solution 2 — surcharger l'opérateur < pour « faciliter le tri ». La réponse la plus orientée objet consiste à définir dans la classe la méthode spéciale `__lt__` (*less than*). C'est elle que Python appelle lorsqu'il compare deux objets avec `<`, donc lorsqu'il trie une liste :

```
def __lt__(self, autre):
    """Ordre : moyenne décroissante, puis nom croissant."""
    if self.moyenne() != autre.moyenne():
        return self.moyenne() > autre.moyenne()
    return self.nom < autre.nom
```

Une fois `__lt__` définie, la liste devient directement triable : il suffit d'écrire `L.sort()` (ou `sorted(L)`) **sans aucun argument**. C'est exactement le sens de la question « comment faire pour faciliter le tri de cette liste ? » : on encapsule le critère de tri dans la classe elle-même.

Programme de test complet :

```

e1 = Etudiant(1, "Alami", "Sara")
e2 = Etudiant(2, "Berrada", "Omar")
e3 = Etudiant(3, "Alaoui", "Yasmine")

notes1 = {"Res": 15, "Admin": 12, "Bd": 14, "Web": 13,
          "tn": 16, "ang": 11, "com": 14, "prog": 15}
notes2 = {"Res": 12, "Admin": 10, "Bd": 11, "Web": 14,
          "tn": 13, "ang": 15, "com": 12, "prog": 10}

for m, n in notes1.items():
    e1.addNote(m, n)
for m, n in notes2.items():
    e2.addNote(m, n)
for m, n in notes1.items():      # mêmes notes que e1 :
    e3.addNote(m, n)            # e3 aura la même moyenne que e1

L = [e1, e2, e3]
L.sort()                        # utilise __lt__
for e in L:
    print(e.nom, "-", round(e.moyenne(), 2))

```

Affichage obtenu :

```

Alami - 13.82
Alaoui - 13.82
Berrada - 12.44

```

On vérifie les deux critères : Alami et Alaoui ont la même moyenne (13.82) et sont donc classés par ordre alphabétique ; Berrada, dont la moyenne est plus faible, apparaît en dernier.

À retenir

Un **attribut d'instance** (créé via `self` dans `__init__`) est propre à chaque objet ; un **attribut de classe** est partagé par toutes les instances et se prête aux données communes, comme les coefficients des modules. Pour **valider les données à la création**, on lève une exception (`raise ValueError(...)`) dans le constructeur : l'objet n'est alors pas créé. Une **méthode de classe** (`@classmethod`, premier paramètre `cls`) s'utilise quand le traitement concerne la classe et non un objet particulier (`estBissextile`, `setCoeff`). Enfin, les **tris** se paramètrent avec `key=` (éventuellement un tuple pour des critères multiples, avec un signe moins pour inverser un ordre numérique), ou se simplifient définitivement en surchargeant `__lt__` dans la classe.